

Customer Solution Blueprint — mermaid-event-model

FY2026 Strategic Accounts · Customer Solution Blueprint

Solution name	Event Model DSL, Renderer & Code-Generation Pipeline (mermaid-event-model)
Solution type	Domain modeling toolchain + reference event-sourced application
Primary repository	patrocinio/mermaid-event-model (fork of howarddierking/mermaid-event-model)
Published artifact	@howarddierking/mermaid-event-model (npm)
Live demo	https://howarddierking.github.io/mermaid-event-model/ · https://d2r5vmr22arud4.cloudfront.net
Deployed AWS account / region	220133863472 / us-east-1
Blueprint date	2026-08-20
Status	Active — reference implementation deployed; toolchain published

Template note. This document follows the standard FY2026 Strategic Accounts Customer Solution Blueprint section structure (Executive Summary → Business Context → Solution Overview → Architecture → Data & Domain Model → Deployment Topology → Security & Governance → Operations → Risks → Roadmap → Appendices). The attached .docx template could not be parsed to its literal headings from the chat attachment; if your copy uses different section names, the content below maps cleanly onto them.

1. Executive Summary

mermaid-event-model is a domain-modeling toolchain built around **Event Modeling** — a technique that describes a system as a strict horizontal timeline of UIs, commands, domain events, read models, and automations laid out in swimlanes (actors on top, aggregates on the bottom, commands and read models on a central time axis).

The solution has three layers that reinforce one another:

1. **A DSL and renderer.** A small, dependency-light DSL (markdown with a fenced eventModel / sliceTests block) plus an SVG renderer that lays the model out as a topologically-ordered timeline. Shipped as an npm package and as a Mermaid external-diagram type.

2. **An executable specification.** Each vertical *slice* of behavior is captured as a `sliceTests` Given/When/Then spec — concrete, field-level, example-valued — so the model doubles as a behavioral contract, not just a picture.
3. **A code-generation pipeline.** Slice specs drive generation of runnable, event-sourced application code against multiple targets (AWS-native serverless, Axon, Rust), all from one stack-independent blueprint. A reference hotel-booking system is generated and deployed to AWS to prove the round trip.

Why it matters for a strategic account. The solution addresses the perennial gap between *how a system is described* and *how it actually behaves*. The model, the tests, and the deployable code all derive from a single source of truth and travel together as the system evolves. It validates both axes of "is the architecture right" — *structure* (the topology of nodes and causal flows) and *behavior and correctness* (slices expressed as Given/When/Then).

Current state. The renderer and authoring skills are published; a reference hotel-booking domain is modeled in three styles (aggregate-based, DCB, fan-in), sliced, spec'd, and code-generated; and a working AWS topology (event store + regional primary + a SageMaker occupancy forecast + a Rust register Lambda) is deployed behind a CloudFront static site.

2. Business Context

2.1 Problem statement

Teams routinely maintain three drifting representations of the same system: architecture diagrams, test suites, and production code. Diagrams rot, tests encode assumptions no one can trace back to intent, and the mapping between "the box on the whiteboard" and "the Lambda in production" is tribal knowledge. Onboarding, audits, and change-impact analysis all pay the tax.

2.2 Solution thesis

Make the **model** the artifact of record, make the **slice spec** the behavioral contract, and **generate** the code. Because layout is a true topological order of the causal edges, the diagram is not decorative — the horizontal position of every node is its earliest possible time given the declared causality, which is the core invariant an Event Model must hold.

2.3 Target domain (reference)

A **hotel booking system**: guest registration, room inventory, availability, booking, check-in/out, housekeeping, payment processing (with an external gateway), and sales reporting. The domain is intentionally rich enough to exercise every modeling construct: fan-in read models, feedback cycles, external events, automations, and translation slices.

2.4 Stakeholders

- **Domain modelers / architects** — author and evolve the Event Model.
- **Engineers** — consume generated handlers and slice tests; implement bodies where generation stops.
- **QA / audit** — read slice specs as the canonical behavioral record.
- **Platform / SRE** — own the deployed AWS topology and its cost/operational posture.

3. Solution Overview

3.1 Capabilities

Capability	Description	Where it lives
Event Model DSL	Actors, aggregates, UIs, commands, domain/external events, read models, automations, slices, typed data sections, tag axes	<code>event-model.js</code> (<code>parse/rank/layout/draw</code>)
SVG renderer	Topological column layout, swimlanes, collapsible data/reads sections	<code>event-model.js</code> , <code>event-model-mermaid.js</code>
Slice tests	Given/When/Then behavioral specs sharing the model's visual vocabulary; <code>error[...]</code> and <code>none[...]</code> outcomes	<code>slice-tests.js</code> , <code>slice-tests-mermaid.js</code>
DCB modeling	Consistency boundaries via <code>reads [...]</code> by <code><axis></code> instead of aggregates	<code>blueprint_dsl_dcb.md</code> + parser support
Authoring skills	Claude Code plugin: <code>event-model</code> , <code>add-slices</code> , <code>spec-slices</code> , <code>add-tests</code> , <code>validate-completeness</code> , <code>create-event-model</code>	<code>skills/</code> , <code>.claude-plugin/plugin.json</code>
Code generation	Slice spec → runnable event-sourced code for AWS / Axon / Rust, optionally via a stack-independent core	<code>codegen.js</code> (+ Rust target)
Reference deployment	Hotel event store, regional primary, SageMaker forecast, Rust Lambda, static site	AWS acct <code>220133863472</code>

3.2 The two diagram kinds

- **eventModel** — the structure: what elements exist and how they causally flow (`ui` → `command` → `domainEvent` → `readModel` → (`ui` | `automation`)).
- **sliceTests** — the behavior: one Given/When/Then card per test, reusing the same element vocabulary, with two spec-only kinds — `error[<code>] ["message"]` (an asserted rejection) and `none["message"]` (an asserted empty query result).

3.3 Vertical slices as the unit of work

A **slice** is the smallest set of edges that must change together — the blast radius of a change. The four canonical patterns are **Command**, **View**, **Automation**, and **Translation** (the last two structurally identical, discriminated only by the presence of an `externalEvent`). Slices are the unit of *specification*; generated handlers are the unit of *regeneration*.

4. Architecture

4.1 Toolchain architecture (no build step)

The renderers are plain ES modules that take `d3` (and optionally `mermaid`) as peer dependencies. The parse → rank → layout → draw pipeline:

1. **Parse** — `parseEventModel(src)` → `{ actors, aggregates, elements, edges, slices }`; elements carry optional `fields` and `reads`.
2. **Rank** — `computeRanks` does a DFS to find back-edges (so feedback cycles like `paymentSubmitted` ↔ `paymentsToProcess` don't blow up), then Kahn topological sort with declaration order as tiebreaker; every element gets a unique column. `reads` is ignored here (it is a hydration directive, not a flow edge).
3. **Layout + draw** — `layoutEventModel` stacks lanes (optional `External` on top → actors → `Time` → aggregates → optional `Events` on the bottom) and `renderEventModel` draws bands, a dashed time axis, bezier edges, and collapsible multi-section nodes.

4.2 Architecture-as-code primitives

The Event Model vocabulary is itself an architecture-as-code representation: every construct is a typed, machine-readable primitive that the toolchain parses, validates, and generates from.

Event Model concept	Architectural role	Notes
Actor / aggregate (swimlane)	node	Actors are actor-type nodes; aggregates are service/bounded-context nodes.
Command, read model, automation	node (service/process)	Deployable/runnable units in the generated topology.
Flow edge (-->)	relationship (connects/interacts)	Causal edges become directed relationships.
Domain/external event	contract crossing a boundary	The event is the interface between producer and consumer.
Data section (typed fields)	interface schema	Field types map to message schemas.
DCB reads [...] by <axis>	consistency control	The consistency boundary is a governable, declarative constraint.
Slice	pattern	A reusable, validated grouping of nodes + relationships.
Slice test (sliceTests)	behavioral contract	The correctness layer — Given/When/Then over the structure above.

Governance angle. Because the model is structured data, the same core that drives code generation can also emit a topology manifest for validation in CI — checking that what is deployed matches what was modeled, and closing the loop between "the model" and "what is actually running" (see §10).

4.3 Deployed application architecture (reference)

The generated hotel system is event-sourced: commands validate against replayed events, emit domain events to an append-only store, and projections build read models consumed by UIs and automations. External events (position updates, gateway confirmations) enter through translation slices.

5. Data & Domain Model

5.1 Two modeling styles, one domain

- **Aggregate-based** (`blueprint_dsl.md`): events are qualified to aggregates (`Inventory` , `Auth` , `Payment`). Consistency is per-aggregate.
- **Dynamic Consistency Boundary (DCB)** (`blueprint_dsl_dcb.md`): no aggregates. Each command declares the past event types it must replay via `reads [...] by <axis>` . A **tag axis** (`*field`) is an independent handle a decision scopes its boundary to; axes on events never compose into a key, but on a read model multiple `*` do compose (a row is one thing). A command's boundary is an **OR of branches**, one per `reads` clause; `by [a, b]` AND's axes within a branch.

This dual expression is a strength: the same business domain is shown under two consistency philosophies, letting an account compare aggregate-oriented and DCB-oriented designs on identical behavior.

5.2 Representative slices (reference domain)

- **Command:** Register, Add Room, Book Room, Check-in, Ready Room, Pay, Process Payment.
- **View:** Room Availability, Cleaning Schedule, Guest Roster, Payments to Process (fan-in of `paymentRequested + paymentSubmitted + paymentSucceeded`), Sales Report.
- **Automation:** Check-out Automation, Payment Processor, Availability Maintainer.
- **Translation:** Hotel Proximity Translator (external `positionUpdated` → `guestLeft`), Gateway Confirmation (external `gatewayConfirmed` → `paymentSucceeded`).

5.3 Behavioral contracts

Each slice's `## Tests` section holds Given/When/Then specs with concrete example values (`checkIn: date = 2026-08-12`) that double as generation fixtures. Two outcome kinds carry governance weight:

- `error[<code>] ["message"]` — an asserted rejection; generation maps the code to a domain exception type (`guest-already-registered` → `GuestAlreadyRegistered`) and emits the message verbatim.
- `none["message"]` — an asserted empty result for a View query (e.g. a room search past the seeded availability horizon must return *no* rooms — the case a naive projection gets wrong).

5.4 Information completeness

The `validate-completeness` skill traces every field in every UI and read model backward through events and commands to confirm no data is assumed or missing — a static guarantee that the model can actually produce what its screens display.

6. Code Generation Pipeline

- **Input:** a slice spec (`Model + Description + Tests`), the canonical per-slice record.
- **Core:** an optional stack-independent "manifest core" derived from the model, from which targets are generated (`--via-core`).
- **Targets:** AWS-native serverless, Axon (JVM), and Rust.
- **CLI (from prior toolchain work):** `node bin/gen.js <slice.md> --target aws|axon|rust [--via-core] [-o file]`.
- **Round trip:** the same slice spec produces the behavioral tests *and* the handler skeletons, so the test asserting a rejection and the code throwing it are one mechanical mapping apart.

This is the crux of the value proposition: **one blueprint, many runtimes**, with the behavioral spec regenerated alongside the code.

7. Deployment Topology

7.1 In-repo infrastructure-as-code (present today)

- `infra/lib/static-site-stack.ts` — AWS CDK stack for the **static demo site**: S3 bucket + CloudFront distribution + a GitHub Actions deploy role.
 - Distribution: `https://d2r5vmr22arud4.cloudfront.net` (ID `E3CLX8H26HD0TV`)
 - Bucket: `mermaideventmodelsite-sitebucket397a1860-ka1nm4ewv9dc`
 - Deploy role: `MermaidEventModelSite-GitHubActionsDeployRole`
- `aws-app/sagemaker/` — SageMaker assets for the occupancy forecast.
- `rust-app/` — Rust Lambda build artifacts (`Cargo.lock` , `target/`).

The event-store and regional-primary CDK sources are not co-located in this repo snapshot; they were deployed from the generation workspace. The resources below reflect the deployed state recorded during that work.

7.2 Deployed AWS resources (account `220133863472` , `us-east-1`)

Stack / resource	Contents
<code>HotelEventStore</code>	Append-only event store (DynamoDB <code>HotelEvents</code> + 5 GSIs)
<code>HotelRegionalPrimary</code>	VPC (1 NAT), Redis, Kinesis, DynamoDB, Lambdas, API Gateway
<code>hotel-occupancy-forecast</code>	SageMaker serverless endpoint (scales to zero)
<code>hotel-register-rust</code>	Rust <code>register</code> Lambda (per-invoke cost)
<code>MermaidEventModelSite</code>	CloudFront + S3 static site (see §7.1)

7.3 Deploy / teardown

- Static site: `./deploy.sh deploy` .
- Rust build (per shell): `export PATH="$HOME/.cargo/bin:$PATH"` then `cd rust-app && node emit.mjs && cargo lambda build --release --arm64` .
- **Teardown**: `aws-app/teardown.sh` , then `cd rust-app/cdk && npx cdk destroy HotelRustRegister` .

8. Security & Governance

- **Static-site delivery** is CloudFront-fronted; the S3 origin is not served directly. GitHub Actions deploys via a scoped IAM role rather than long-lived keys.
- **Event-sourced audit trail**. An append-only event store is inherently auditable — every state change is a recorded fact.

- **Behavioral governance.** Slice specs are the canonical, reviewable statement of intent; `error / none` assertions make rejection and empty-state behavior explicit and testable.
 - **Structural governance (recommended).** Validate the *topology* the generator emits: emit the generated nodes/relationships/interfaces as a machine-readable manifest and validate it in CI against the deployed AWS topology, and treat DCB consistency boundaries as declarative controls. This gives the account a single pipeline gate covering both structure and behavior (slice tests).
 - **Least privilege / production safety.** Treat the deployed account as production-adjacent: prefer read/describe operations, and require explicit confirmation before any destructive change (see §9.2).
-

9. Operations

9.1 Runtime posture

- SageMaker endpoint is **serverless** and scales to zero — no idle inference cost.
- Rust register Lambda is **per-invoke** — no idle cost.
- The static site is effectively fixed, low cost (CloudFront + S3).

9.2 Cost & billing watch ⚠️

The **NAT Gateway** and **Redis** in `HotelRegionalPrimary` run **24/7** and bill continuously whether or not traffic flows. These are the dominant standing costs. If the reference environment is idle, tear down `HotelRegionalPrimary` (or at least the NAT + Redis) to stop the meter. SageMaker (scales to zero) and the Rust Lambda (per-invoke) do not carry idle cost.

9.3 Local development

- No JS build step; run demos with `python3 -m http.server 8000` then open `model-viewer.html` (defaults to `blueprint_dsl_dcb.md`, hot-reloads every 1.5s).
 - No JS unit-test harness; validate parser/layout changes via a Node one-liner or by rendering the relevant `blueprint_*.md`.
-

10. Risks & Mitigations

Risk	Impact	Mitigation
Idle AWS spend (NAT + Redis 24/7)	Ongoing cost with no usage	Tear down <code>HotelRegionalPrimary</code> when idle; alarm on monthly spend
laC not fully co-located in repo	Reproducibility gap for event-store/regional stacks	Consolidate all CDK sources into the repo; pin them to the generator output
No automated JS test suite	Renderer/parser regressions	Add minimal Node-based parse/layout smoke tests to CI
Model ↔ deployment drift	"Diagram doesn't match prod" recurs at the topology level	Emit + validate a topology manifest in CI (§4.2, §8)
Generation stops at skeletons	Hand-written bodies can diverge from specs	Keep slice tests as the gate; regenerate handlers, not hand edits
Fork/upstream divergence	Confusion over the source of truth	Keep <code>patrocinio</code> fork rebased on <code>howarddierking</code> ; scope PRs per feature

11. Roadmap

1. **Topology validation.** Emit a machine-readable topology manifest from the manifest core; validate the deployed AWS topology against it in CI. Treat DCB boundaries as declarative controls.
2. **Consolidate laC.** Bring the event-store and regional-primary CDK sources into the repo so the whole topology regenerates from one place.
3. **CI behavioral gate.** Execute generated slice tests against generated handlers in the pipeline.
4. **Cost automation.** Scheduled teardown/rebuild of the standing regional stack, or a NAT-less/serverless-cache variant to remove idle cost.
5. **Additional generation targets** as account needs dictate, all from the same stack-independent core.

12. Appendices

12.1 Key repository files

Path	Purpose
event-model.js / event-model-mermaid.js	Core renderer + Mermaid adapter (eventModel)
slice-tests.js / slice-tests-mermaid.js	Core renderer + Mermaid adapter (sliceTests)
index.js , package.json	npm entry points (default export registers both diagram types)
blueprint_dsl.md	Aggregate-based hotel model
blueprint_dsl_dcb.md	DCB-style hotel model (reads ... by <axis>)
blueprint_dsl_fanin.md	Fan-in stress test (16 events → one read model)
blueprint_sliceTests.md	Slice-test reference (four canonical patterns)
blueprint_dsl_dcb-slices/ , blueprint_dsl_fanin-slices/	Per-slice spec files
codegen.js	Code generation (AWS / Axon / Rust; optional stack-independent core)
infra/lib/static-site-stack.ts	CDK for the CloudFront/S3 demo site
skills/ , .claude-plugin/plugin.json	Authoring skills + Claude Code plugin manifest

12.2 Canonical flow pattern

ui → command → domainEvent → readModel → (ui | automation)

12.3 References

- Event Modeling — <https://eventmodeling.org>
- Live demo — <https://howarddierking.github.io/mermaid-event-model/>
- Deployed site — <https://d2r5vmr22arud4.cloudfront.net>